OPTIMIZATION OF PEARSON AND BLUR

DV1674 | ASSIGNMENT 2

Alexander Järnström
aljr21@student.bth.se

Jesper Lennehag
jele22@student.bth.se

Contents

I. Introduction

We got two programs, pearson and blur, both running different core algorithms with different performance issues. The purpose of this work is to optimize them improving their system utilization and time frame, performing tests to outline a baseline for comparison, and document the changes and their results.

II. Method

Each program was tested to determine a baseline using several profiling tools like:

- Valgrind: Base tool
  - Callgrind: Used for cache simulation and viewing instruction trees.
  - Massif: Used for memory utilization and analysis.
  - Memcheck: Used to check for leaks.
- Perf stat: used to gather system information during runs. Used with `-d` for cache information and `--repeat 10` to run the test ten times and give an average.
- VTune: sanity checks.

After a baseline was determined different optimisation steps were taken, after which performance tests were run. The improvement would be removed if it worsened the performance.

Parallelism will be implemented into the program when the sequential version is optimal.

III. Pearson

The optimization process followed a simple process: run tests, implement optimization, run tests and determin if they should be kept. Multithreading was implemented using `pthread` when the sequential version reached an optimal stage.

III.I. Sequential

The initial step taken was to enable compiler optimizations with the flags `-O2` and `-O3` where the one with best time was kept, `-O3` proved to be 527ms quicker than `-O2` which gave the results in Table 1. All the other stages are using the `-O3` flag when compiling.

Table 1: Speed ups after using compile time optimizations.

Sizes	Speed Up
128	2.911
256	3.466
512	4.284
1024	4.711

Next was the implementation of a cache, keeping all the results from the `mean` and `magnitude` methods in the `Vector` class. This was achieved by creating two arrays `mean_cache` and `magnitude_cache` which were initialized with the maximum value a double can take. It would check the cache, if the value equals the maximum a value for mean and magnitude would be calculated and then stored at the provided index, otherwise the value would be collected and used. This gave a speed up between 3.4 and 8.2 depending on the vector size, Table 2.

Table 2: Speed up after implementing a cache.

Sizes	Speed Up
128	3.41
256	4.901
512	6.718
1024	8.223

We continued to loop unroling, allowing the loops in the vector to do four calculations per iteration and eliminating dependencies by doing said calculations to temporary variables and then calculating the final result from them minimizing the pipeline stalls. This simple change give the speed ups show in Table 3.

Table 3: Speed up after implementing loop unroling.

Sizes	Speed Up
128	3.438
256	6.053
512	11.693
1024	21.02

The third step took inspiration from locality, moving the calculations into the vector. `VTune` showed massive allocation amounts, for size 1024 the total reached 30.2 GB, where the biggest corporate is the constant copying and construction of new vectors when performing all the calculations. This step took the cache and *moved* it into the vector, or with other words replaced it with the vector itself. By realizing the original vector wasn't needed, all the calculations were changed to be preformed directly on the original. This replaced the original values by the calculated ones storing them in the vector, saving them for the `dot` method. This step lowered the allocation amount down to 103.9 MB for 1024, meaning the requests for more memory to the OS fell giving a speed up of 3.43 for 128 and 21 for 1024, Table 4.

Table 4: Speed up after implementing the locality optimizations.

Sizes	Speed Up
128	4.163
256	7.698
512	13.966
1024	23.147

The fifth step focused on IO the functions taking the most time now were `dot`, `read`, and `write`. `write` is heavily IO bound due to reading and writing file content to the disk, `write` was optimized by removing `std::endl` from the stream and replacing it with `\n` this allowed the stream to continue without clearing the buffer every iteration through the results. This gave the final speed up of 9.98 for 128 and 57.048 for 1024, Table 5.

Table 5: Changes between loop unfolding and the IO step for size 1024.

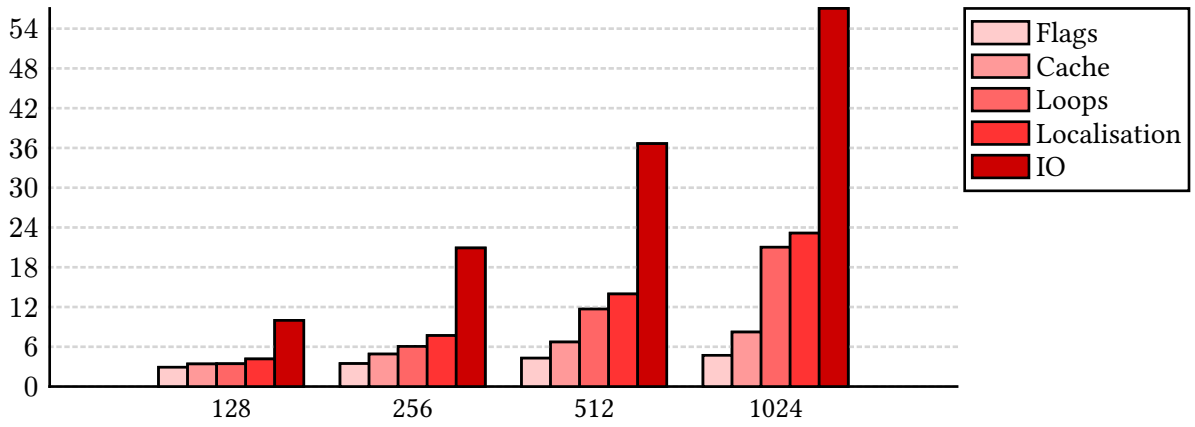
Sizes	Speed Up
128	9.979
256	20.907
512	36.651
1024	57.048

All the changes are illustrated in Graph 1 showing all speed ups through out the steps on different vector sizes and Table 6 show all recorded times.

III.II. Parallelism

There are some steps needed to achieve parallelism for the pearson program. Firstly a way for the user to specify the amount of wanted threads, secondly split up the work evenly, and lastly a safe way to get the results. To get the thread count a third argument for the binary was specified, thread count, which is used to separate the data evenly and create the wanted threads.

Separating the workload proved to be more of a hassle than expected, the `dot` method isn't run between



Graph 1: Illustration of all the different sizes through all the taken steps.

all possible column pairs in the matrix, instead every column pairs with every column in front it. This can be visualized as *pair triangle*, meaning we can calculate the calculation count using:

$$\text{pair count} = \frac{\text{vector size} * (\text{vector size} - 1)}{2}$$

, due to the last vector not being multiplied to it self, one is removed from it. The result of the multiplication will always be a multiple of two, thanks to the vector size. We then calculate the pair segment size which will be used to separate them between the threads:

$$\text{segment size} = \frac{\text{pair count}}{\text{thread count}}$$

This is we needed to execute the `dot` method evenly across all the threads. But this excludes the `prepeare` method, leaving potential for parallelism. We continued to separate the dataset between the threads which proved to be simpler than the previous one, the only issue was the dependency between `dot` and `prepeare`. `dot` need `prepeare` to run on both the vectors in the calculation before executing. Giving us two choices: (1) running `prepeare` every time `dot` needed it, skipping the calculation if it was done, or (2) run `prepeare` on all the vectors before continuing. Due to all the needed mutex locks for the first option to mitigate all the race conditions, option two was chosen.

In conclusion, the program first separates the dataset between the threads, then it starts the threads which run the `prepeare` method on each vector. The main thread continues to add the vectors to pairs giving them their index for the result and then stops at a `pthread_barrier`, when all the other threads are done processing they'll reach the same barrier, which will let them proceed when all threads reach it. The threads continue to process the `dot` method and places the result in the result vector at the given index, they'll join the main thread when the processing is done.

The same tests were run at the same sizes for each thread count, 1 to 32, giving the results shown through Figure 1 and Figure 2.

- Size 128 didn't showed any improvement through out the threads, hovering between 0.8 and 0.9 speed up. This isn't better than the sequential version, meaning a this size threads aren't worth it.
- Size 256 showed slight improvement, bearly reaching a speed up at two and four threads.
- Size 512 shows a slightly better situation than the other two, with a result of 1.129x speed up.
- Size 1024 gave the best improvement, with the greatest speed up of 1.399x with eight threads. This also giva the biggest speed up compared to the base version with a value of 79.8 times.

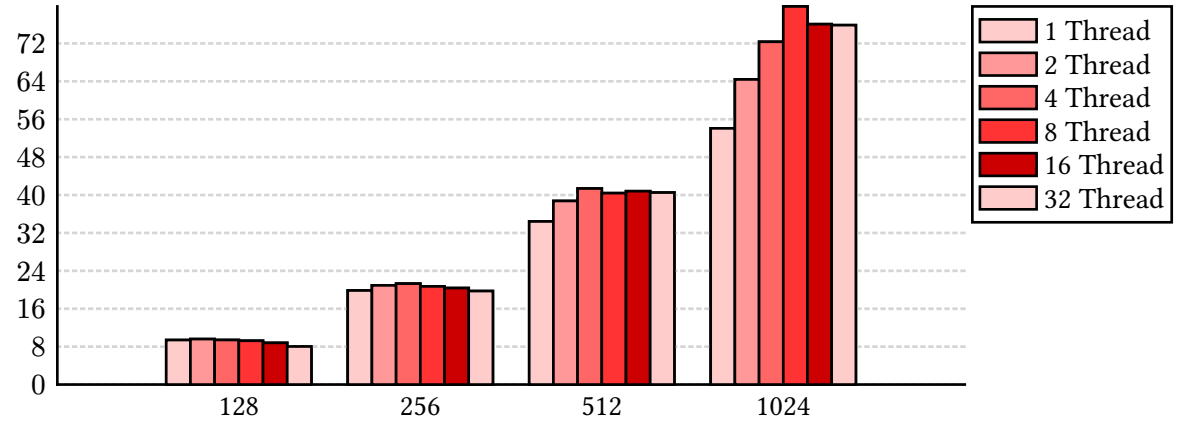
All the vector sizes show a negative impact when only running one thread, Figure 2, which is explained by the extra processing needed to prepare the data for multi-threading.

III.III. Performance and Scalability

The speed up grow with the thread count if the vector size is big enough, Figure 2. This allow the CPU utilization to grow with the data with greater speed ups

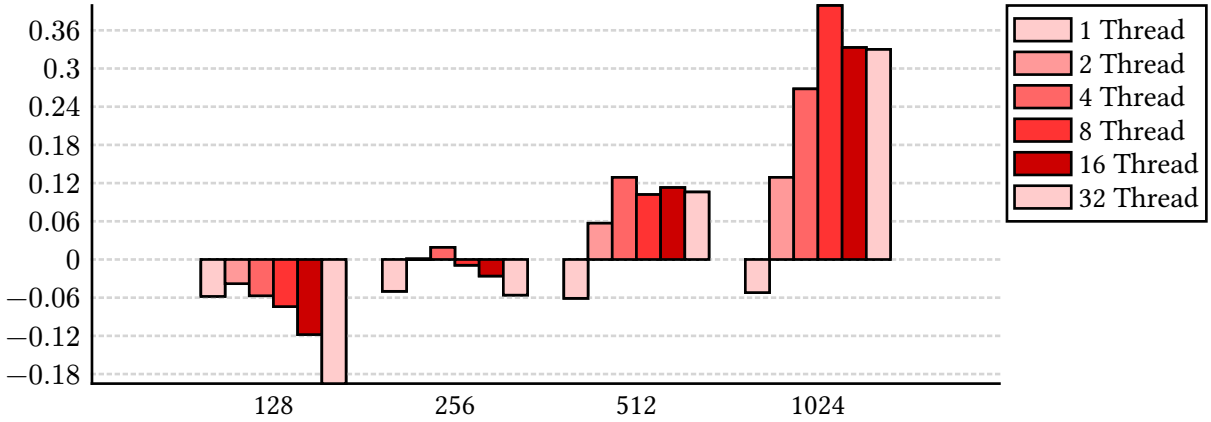
Table 6: All run times in seconds.

Sizes	Base	Flags	Cache	Loop	Locality	IO
128	0.053645	0.018428	0.01573	0.015603	0.012885	0.005376
256	0.367857	0.106119	0.075052	0.060773	0.047789	0.017595
512	2.698143	0.629849	0.40162	0.23075	0.193188	0.073618
1024	20.61275	4.375323	2.506641	0.98062	0.890509	0.36132



Sizes	1	2	4	8	16	32
128	9.398	9.598	9.411	9.244	8.8	8.034
256	19.853	20.935	21.3	20.72	20.371	19.735
512	34.422	38.752	41.376	40.402	40.81	40.521
1024	54.077	64.398	72.346	79.807	76.065	75.856

Figure 1: Total speed up per thread count compared to base sequential version.



Sizes	1	2	4	8	16	32
128	−0.058	−0.038	−0.057	−0.074	−0.118	−0.195
256	−0.05	0.001	0.019	−0.009	−0.026	−0.056
512	−0.061	0.057	0.129	0.102	0.113	0.106
1024	−0.052	0.129	0.268	0.399	0.333	0.33

Figure 2: Total speed up per thread count compared to optimized sequential version subtracted by one to highlight the difference.

than shown here. The memory utilization grow with the vector size, Figure 3, but not the thread count. This allow us to use as many threads as we want with minimal memory waste, only limited by the growing vector.

III.IV. What more?

Vectorisation using libraries such as `immintrin.h` could improve CPU utilization, performing several calculations simultaneously. This could give performance improvements, though the data sizes in this project seem to be too small to gain enough time to cover for the preparations. Moving the thread preparations to the `read` function could improve some overhead, the function itself could probably be improved with some other file reading or double conversion utility, though both `read` and `write` are heavily IO bound.

IV. Blur

The objective of this work was to optimize and parallelize the Gaussian blur filter while preserving correctness. The original implementation applies a separable Gaussian blur using two passes: a horizontal pass fol-

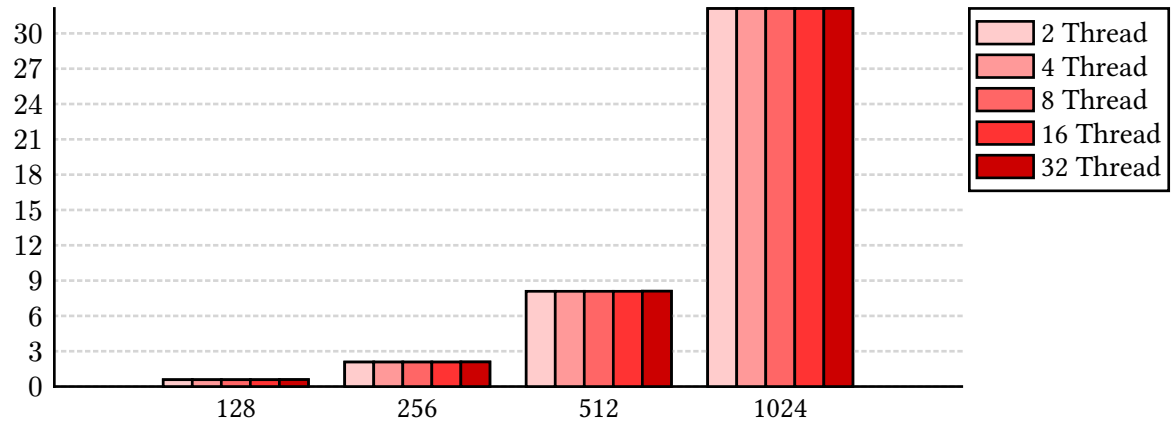


Figure 3: Maximum memory consumption during run time per thread count.

lowed by a vertical pass. Although the algorithm is already more efficient than a full two-dimensional convolution, several implementation-level inefficiencies remained.

The optimization process was performed incrementally. After each optimization step, performance measurements were collected and compared against the original implementation. Once the sequential optimizations had been completed, the filter was parallelized using POSIX threads (pthreads) to utilize multiple CPU cores.

Correctness was verified after each optimization and after parallelization using the provided verification script.

IV.I. Sequential Optimizations

IV.I.I. Precomputation of Gaussian Weights

In the original implementation the Gaussian weights were recomputed for every pixel in both the horizontal and vertical passes. Since the weights depend only on the blur radius, recalculating them for each pixel introduces unnecessary overhead.

To eliminate this redundancy the weight array was computed once before the blur operation and reused throughout both passes.

This optimization significantly reduced the number of expensive exponential function evaluations and produced a speedup of approximately 1.4 - 1.6x compared to the baseline implementation.

Table 7: Speedup achieved by precomputing Gaussian weights relative to the original implementation.

Image	Speedup
im1	1.45×
im2	1.39×
im3	1.49×
im4	1.57×
Average	**1.48×**

IV.I.II. Direct Memory Access

The original implementation accessed image data through matrix accessor functions such as `r(x,y)`, `g(x,y)`, and `b(x,y)`. These function calls were executed repeatedly inside the innermost loops of the blur algorithm.

To reduce this overhead, direct pointers to the red, green, and blue channel arrays were obtained using `get_R()`, `get_G()`, and `get_B()`. Pixel values could then be accessed directly through array indexing.

Removing the accessor function calls reduced instruction overhead and allowed the compiler to generate more efficient code. This optimization increased the overall speedup to approximately 2.4 - 2.7x relative to the original implementation.

Table 8: Speedup achieved by direct memory access relative to the original implementation.

Image	Speedup
im1	2.56×
im2	2.39×
im3	2.46×
im4	2.66×
Average	**2.52×**

IV.I.III. Cache-Friendly Linear Traversal

The original implementation traversed the image using nested x–y loops. The optimized implementation instead processes pixels using a single linear index corresponding to the image's memory layout.

Since image data is stored contiguously in memory, linear traversal improves spatial locality and enables more effective hardware prefetching. As a result, cache utilization improves and memory accesses become more efficient.

The performance gain from this optimization was smaller than the previous optimization. This is likely because neighboring pixels were already accessed during the blur operation, resulting in relatively good cache behavior before the optimization was applied.

Table 9: Speedup achieved by cache-friendly linear traversal relative to the original implementation.

Image	Speedup
im1	2.52×
im2	2.50×
im3	2.37×
im4	2.69×
Average	**2.52×**

IV.I.IV. Final

The largest sequential improvement was obtained from direct memory access. The linear traversal optimization provided only a modest additional gain because neighboring pixels were already stored close together in memory, resulting in relatively good cache utilization before the optimization.

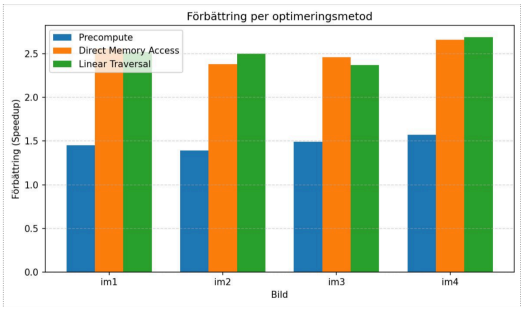


Figure 1: Speedup obtained after each sequential optimization step.

IV.II. Parallelization Strategy

IV.II.I. Work Partitioning

After completing the sequential optimizations, the blur filter was parallelized using POSIX threads. The image was partitioned by rows, and each thread was assigned a contiguous range of rows to process.

The number of rows assigned to each thread was calculated as:

$$\text{rows\_per\_thread} = \frac{\text{image\_height}}{\text{thread\_count}}$$

This partitioning strategy provides good load balancing because each pixel requires approximately the same amount of computation.

IV.II.II. Data Organization

The image is stored using three separate arrays corresponding to the red, green, and blue color channels. Additional scratch arrays are used to store intermediate results generated during the horizontal blur pass.

All threads share access to these arrays. Since each thread writes only to its assigned rows, no write conflicts occur during execution.

IV.II.III. Synchronization

The blur operation consists of two dependent phases:

1. Horizontal blur
2. Vertical blur

The vertical pass requires all intermediate values from the horizontal pass to be available before execution can begin. To ensure correctness, a pthread barrier was introduced between the two phases.

After finishing the horizontal pass, all threads wait at the barrier. Once every thread reaches the barrier, execution continues and the vertical pass begins.

This synchronization mechanism guarantees that no thread reads incomplete data from the scratch buffers.

IV.III. Performance and Scalability

The multithreaded implementation achieved near-linear speedup up to approximately eight threads. Figure 2 shows the relationship between speedup and thread count for the tested images.

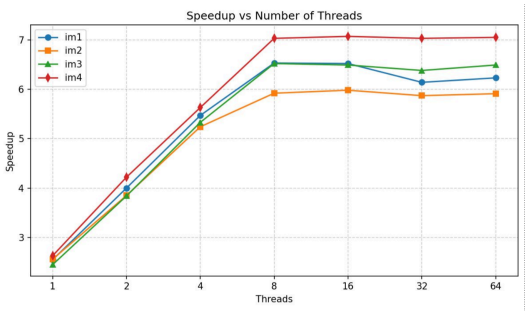


Figure 2: Speedup as a function of the number of threads.

The results show that speedup increases steadily from one to eight threads, reaching approximately 6–7x

depending on the image. Beyond eight threads, performance improvements become negligible and, in some cases, slightly decrease.

Several factors contribute to this behavior:

1. The available CPU cores are fully utilized at approximately eight threads.
2. Memory bandwidth becomes a limiting factor because each thread continuously reads and writes large image buffers.
3. Synchronization overhead increases as the number of threads grows.
4. According to Amdahl's Law, the serial portion of the program limits maximum achievable speedup.

As a result, additional threads beyond eight provide little additional performance.

IV.IV. Additional Optimization Opportunities

IV.IV.I. SIMD Vectorization

Modern CPUs support Single Instruction Multiple Data (SIMD) instruction, which allow multiple pixel values to be processed simultaneously. Since the Gaussian blur performs the same arithmetic operations on large arrays of pixel data it is well suited for vectorization. Using SIMD intrinsics or compiler-assisted vectorization could further reduce execution time by increasing the amount of work performed per CPU cycle.

IV.IV.II. Loop Unrolling

The convolution loops could be manually unrolled to reduce loop-control overhead and increase instruction-level parallelism. Unrolling decreases the number of branch instructions executed and may allow the compiler to generate more efficient code. Although modern compilers perform some automatic loop unrolling, additional performance gains may be possible through manual tuning of the most frequently executed loops.

IV.IV.III. GPU Acceleration

Gaussian blur is a highly parallel image-processing operation and is therefore well suited for execution on graphics processing units (GPUs). A GPU implementation using CUDA or OpenCL could process thousands of pixels concurrently, providing substantially higher throughput than a CPU-based implementation. This approach would be particularly beneficial for large images where the overhead of transferring data to and from the GPU is outweighed by the increased computational performance. These optimizations were not explored in this project, but they represent promising directions for future work and could potentially provide additional performance improvements beyond those achieved with the current CPU implementation.